

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI STRUKTUR DATA
DOUBLE LINKED LIST

Disusun Oleh : Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi, S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 6 dengan judul “Impelementasi Struktur Data Double LinkedList” ini dengan baik. Pada laporan praktikum pekan satu ini akan membahas mengenai penerapa konsep ADT (Abstract Data Type) membahas Struktur data LinkedList spesifiknya penerapan Double LinkedList.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Avero yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami struktur data Double LinkedList dan penerapannya dalam java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.	1
1.2 Tujuan Praktikum.	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 NodeDLL	2
2.1.1 Penjelasan Program.....	2
2.1.2 Kode Program	2
2.2 InsertDLL	3
2.2.1 Penjelasan Program.....	3
2.2.2 Kode Program dan Output	3
2.3 Penelurusan DLL	5
2.3.1 Penjelasan Program.....	5
2.3.2 Kode Program dan Output	5
2.4 HapusDLL.....	7
2.4.1 Penjelasan Program.....	7
2.4.2 Kode Program dan Output	7
BAB III KESIMPULAN.....	9
TUGAS PRAKTIKUM.....	10

BAB I

PENDAHULUAN

1.1 Latar Belakang.

Dalam pengembangan perangkat lunak, efisiensi pengelolaan data secara dinamis menjadi hal yang sangat krusial. Struktur data linier seperti array sering kali memiliki keterbatasan dalam hal ukuran yang statis dan proses penyisipan data yang memakan waktu. Double Linked List hadir sebagai solusi yang memungkinkan alokasi memori yang lebih fleksibel, di mana setiap elemen atau simpul memiliki keterhubungan dua arah melalui penunjuk (pointer) ke elemen sebelum dan sesudahnya. Praktikum ini dilakukan untuk mensimulasikan bagaimana data disimpan, dihubungkan, dan dimanipulasi di dalam memori komputer secara efisien.

1.2 Tujuan Praktikum.

Tujuan utama dari praktikum ini adalah untuk memahami konsep dasar dan mekanisme kerja Double Linked List melalui bahasa pemrograman Java. Secara khusus, praktikum ini bertujuan agar praktikan mampu mengimplementasikan pembentukan simpul (node), menguasai teknik penyisipan dan penghapusan data pada posisi tertentu, serta memahami cara melakukan penelusuran data baik secara maju (forward) maupun mundur (backward). Hal ini bertujuan untuk memastikan pemahaman mendalam mengenai logika penanganan alamat memori antar simpul yang saling terhubung.

1.3 Manfaat Praktikum.

Manfaat yang didapatkan adalah kemampuan dalam mengelola memori secara manual melalui pointer, yang merupakan fondasi penting untuk mempelajari algoritma tingkat lanjut. Selain itu, pemahaman ini memberikan dasar yang kuat untuk membangun aplikasi yang membutuhkan navigasi data dua arah secara cepat, seperti sistem daftar putar musik, fitur undo-redo pada perangkat lunak, maupun manajemen antrian pada sistem operasi.

BAB II

PEMBAHASAN

2.1 NodeDLL

2.1.1 Penjelasan Program

Program ini berfungsi untuk mendefinisikan struktur dasar dari sebuah simpul atau Node dalam sistem Double Linked List. Di dalam kelas ini, terdapat tiga komponen utama: variabel data untuk menyimpan nilai bertipe integer, serta dua buah pointer referensi yaitu next yang mengarah ke simpul berikutnya dan prev yang mengarah ke simpul sebelumnya.

2.1.2 Kode Program

```
public class NodeDLL_2511532008 {  
  
    int data_2008;  
    NodeDLL_2511532008 next_2008;  
    NodeDLL_2511532008 prev_2008;  
  
    public NodeDLL_2511532008(int data_2008) {  
  
        this.data_2008 = data_2008;  
        this.next_2008 = null;  
        this.prev_2008 = null;  
    }  
}
```

Gambar 2.1

2.2 InsertDLL

2.2.1 Penjelasan Program

Program ini merupakan implementasi dari berbagai metode manipulasi struktur data untuk menambah elemen baru ke dalam list. Terdapat tiga metode penyisipan utama: insertBegin untuk menambah simpul di posisi paling depan, insertEnd untuk menambah simpul di posisi akhir melalui iterasi hingga ujung list, dan insertAtPosition untuk menyisipkan data pada posisi spesifik di tengah list.

2.2.2 Kode Program dan Output

```
public class InsertDLL_2511532008 {
    static NodeDLL_2511532008 insertBegin_2008(NodeDLL_2511532008 head_2008, int data_2008) {
        NodeDLL_2511532008 new_node_2008 = new NodeDLL_2511532008(data_2008);
        new_node_2008.next_2008 = head_2008;
        if (head_2008 != null) {
            head_2008.prev_2008 = new_node_2008;
        }
        return new_node_2008;
    }

    public static NodeDLL_2511532008 insertEnd_2008(NodeDLL_2511532008 head_2008, int newData_2008) {
        NodeDLL_2511532008 newNode_2008 = new NodeDLL_2511532008(newData_2008);

        if (head_2008 == null) {
            head_2008 = newNode_2008;
        }
        else {
            NodeDLL_2511532008 curr_2008 = head_2008;
            while (curr_2008.next_2008 != null) {
                curr_2008 = curr_2008.next_2008;
            }
            curr_2008.next_2008 = newNode_2008;
            newNode_2008.prev_2008 = curr_2008;
        }
        return head_2008;
    }

    public static NodeDLL_2511532008 insertAtPosition_2008(NodeDLL_2511532008 head_2008, int pos_2008, int new_data_2008) {
        NodeDLL_2511532008 new_node_2008 = new NodeDLL_2511532008(new_data_2008);
        if (pos_2008 == 1) {
            new_node_2008.next_2008 = head_2008;
            if (head_2008 != null) {
                head_2008.prev_2008 = new_node_2008;
            }
            head_2008 = new_node_2008;
            return head_2008;
        }
        NodeDLL_2511532008 curr_2008 = head_2008;
        for (int i_2008 = 1; i_2008 < pos_2008 - 1 && curr_2008 != null; ++i_2008) {
            curr_2008 = curr_2008.next_2008;
        }
        if (curr_2008 == null) {
            System.out.println("Posisi tidak ada");
            return head_2008;
        }
        new_node_2008.prev_2008 = curr_2008;
        new_node_2008.next_2008 = curr_2008.next_2008;
        curr_2008.next_2008 = new_node_2008;

        if (new_node_2008.next_2008 != null) {
            new_node_2008.next_2008.prev_2008 = new_node_2008;
        }

        return head_2008;
    }
}
```

Gambar 2.2

```

public static void printList_2008(NodeDLL_2511532008 head_2008) {
    NodeDLL_2511532008 curr_2008 = head_2008;
    while (curr_2008 != null) {
        System.out.print(curr_2008.data_2008 + " <-> ");
        curr_2008 = curr_2008.next_2008;
    }
    System.out.println();
}

public static void main(String[] args_2008) {
    NodeDLL_2511532008 head_2008 = new NodeDLL_2511532008(2);
    head_2008.next_2008 = new NodeDLL_2511532008(3);
    head_2008.next_2008.prev_2008 = head_2008;
    head_2008.next_2008.next_2008 = new NodeDLL_2511532008(5);
    head_2008.next_2008.next_2008.prev_2008 = head_2008.next_2008;

    System.out.print("DLL Awal: ");
    printList_2008(head_2008);

    head_2008 = insertBegin_2008(head_2008, 1);
    System.out.print("simpul 1 ditambah di awal: ");
    printList_2008(head_2008);

    System.out.print("simpul 6 ditambah di akhir: ");
    int data_2008 = 6;
    head_2008 = insertEnd_2008(head_2008, data_2008);
    printList_2008(head_2008);

    System.out.print("tambah node 4 di posisi 4: ");
    int data2_2008 = 4;
    int pos_2008 = 4;
    head_2008 = insertAtPosition_2008(head_2008, pos_2008, data2_2008);
    printList_2008(head_2008);
}

```

Gambar 2.3

```

terminated> insertDLL_2511532008 pada Application; C:\Users\ran...
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

Gambar 2.4

2.3 Penelusuran DLL

2.3.1 Penjelasan Program

Program ini mendemonstrasikan kapabilitas utama dari Double Linked List, yaitu kemampuan untuk melakukan Traversal atau penelusuran data secara dua arah. Metode forwardTransversal digunakan untuk mencetak data mulai dari simpul kepala (head) hingga ke akhir menggunakan pointer next. Sebaliknya, metode backwardTransversal

2.3.2 Kode Program dan Output

```
public class PenelusuranDLL_2511532008 {
    static void forwardTransversal_2008 (NodeDLL_2511532008 head_2008) {
        NodeDLL_2511532008 curr_2008 = head_2008;

        while (curr_2008 != null){
            System.out.print (curr_2008.data_2008 + " <-> ");
            curr_2008 = curr_2008.next_2008;
        }
        System.out.println();
    }

    static void backwardTransversal_2008 (NodeDLL_2511532008 tail_2008) {
        NodeDLL_2511532008 curr_2008 = tail_2008;
        while (curr_2008 != null) {
            System.out.print (curr_2008.data_2008 + " <-> ");
            curr_2008 = curr_2008.prev_2008;
        }
        System.out.println();
    }

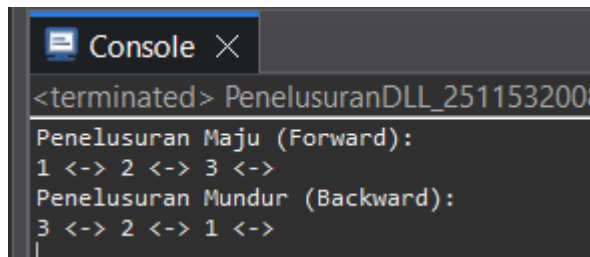
    public static void main (String[] args) {
        NodeDLL_2511532008 head_2008 = new NodeDLL_2511532008(1);
        NodeDLL_2511532008 second_2008 = new NodeDLL_2511532008(2);
        NodeDLL_2511532008 third_2008 = new NodeDLL_2511532008(3);

        head_2008.next_2008 = second_2008;
        second_2008.prev_2008 = head_2008;
        second_2008.next_2008 = third_2008;
        third_2008.prev_2008 = second_2008;

        System.out.println("Penelusuran Maju (Forward):");
        forwardTransversal_2008(head_2008);

        System.out.println("Penelusuran Mundur (Backward):");
        backwardTransversal_2008(third_2008);
    }
}
```

Gambar 2.5



```
Console X
<terminated> PenelusuranDLL_251153200
Penelusuran Maju (Forward):
1 <-> 2 <-> 3 <->
Penelusuran Mundur (Backward):
3 <-> 2 <-> 1 <->
```

Gambar 2.6

2.4 HapusDLL

2.4.1 Penjelasan Program

Program terakhir ini berfokus pada manajemen memori dan penghapusan elemen dari struktur list. Terdapat fungsi `delHead` untuk menghapus elemen pertama, `delLast` untuk menghapus elemen terakhir, dan `delPos` untuk menghapus simpul pada urutan tertentu. Secara teknis, proses penghapusan dilakukan dengan cara memutus keterhubungan simpul yang dituju dan menghubungkan kembali simpul sebelum dan sesudahnya (re-linking) agar rantai data tidak terputus. Program ini juga telah dilengkapi dengan pengecekan kondisi list kosong atau list yang hanya memiliki satu simpul untuk mencegah terjadinya null pointer exception.

2.4.2 Kode Program dan Output

```
public class Hapus_DLL_2511532008 {  
    public static NodeDLL_2511532008 delHead_2008(NodeDLL_2511532008 head_2008) {  
        if (head_2008 == null) {  
            return null; }  
        head_2008 = head_2008.next_2008;  
        if (head_2008 != null) {  
            head_2008.prev_2008 = null; }  
        return head_2008; }  
    public static NodeDLL_2511532008 delLast_2008(NodeDLL_2511532008 head_2008) {  
        if (head_2008 == null) {  
            return null; }  
        if (head_2008.next_2008 == null) {  
            return null; }  
        NodeDLL_2511532008 curr_2008 = head_2008;  
        while (curr_2008.next_2008 != null) {  
            curr_2008 = curr_2008.next_2008; }  
        if (curr_2008.prev_2008 != null) {  
            curr_2008.prev_2008.next_2008 = null; }  
        return head_2008; }  
    }  
    public static NodeDLL_2511532008 delPos_2008(NodeDLL_2511532008 head_2008, int pos_2008) {  
        if (head_2008 == null) {  
            return head_2008; }  
        NodeDLL_2511532008 curr_2008 = head_2008;  
        for (int i = 1; curr_2008 != null && i < pos_2008; ++i) {  
            curr_2008 = curr_2008.next_2008; }  
        if (curr_2008 == null) {  
            return head_2008; }  
        if (curr_2008.prev_2008 != null) {  
            curr_2008.prev_2008.next_2008 = curr_2008.next_2008; }  
        if (curr_2008.next_2008 != null) {  
            curr_2008.next_2008.prev_2008 = curr_2008.prev_2008; }  
        if (head_2008 == curr_2008) {  
            head_2008 = curr_2008.next_2008; }  
        return head_2008; }  
    }  
    public static void printList_2008(NodeDLL_2511532008 head_2008) {  
        NodeDLL_2511532008 curr_2008 = head_2008;  
        while (curr_2008 != null) {  
            System.out.print(curr_2008.data_2008 + " <--> ");  
            curr_2008 = curr_2008.next_2008; }  
        System.out.println(); }  
}
```

Gambar 2.7

```

public static void main(String[] args ) {

    NodeDLL_2511532008 head_2008 = new NodeDLL_2511532008(1);
    head_2008.next_2008 = new NodeDLL_2511532008(2);
    head_2008.next_2008.prev_2008 = head_2008;
    head_2008.next_2008.next_2008 = new NodeDLL_2511532008(3);
    head_2008.next_2008.next_2008.prev_2008 = head_2008.next_2008;
    head_2008.next_2008.next_2008.next_2008 = new NodeDLL_2511532008(4);
    head_2008.next_2008.next_2008.next_2008.prev_2008 = head_2008.next_2008.next_2008;
    head_2008.next_2008.next_2008.next_2008.next_2008 = new NodeDLL_2511532008(5);
    head_2008.next_2008.next_2008.next_2008.next_2008.prev_2008 = head_2008.next_2008.next_2008.next_2008;

    System.out.print("DLL Awal: ");
    printList_2008(head_2008);

    System.out.print("Setelah head dihapus: ");
    head_2008 = delHead_2008(head_2008);
    printList_2008(head_2008);

    System.out.print("Setelah node terakhir dihapus: ");
    head_2008 = delLast_2008(head_2008);
    printList_2008(head_2008);

    System.out.print("menghapus node ke 2: ");
    head_2008 = delPos_2008(head_2008, 2);
    printList_2008(head_2008);

}

```

Gambar 2.8

```

<terminated> Hapus_DLL_2511532008 [Java Application] C:\Us
DLL Awal: 1 <--> 2 <--> 3 <--> 4 <--> 5 <-->
Setelah head dihapus: 2 <--> 3 <--> 4 <--> 5 <-->
Setelah node terakhir dihapus: 2 <--> 3 <--> 4 <-->
menghapus node ke 2: 2 <--> 4 <-->

```

Gambar 2.9

BAB III

KESIMPULAN

Praktikum ini menyimpulkan bahwa Double Linked List merupakan struktur data linier yang sangat adaptif dalam mengelola elemen secara dinamis melalui keterhubungan dua arah. Penggunaan pointer ganda (next dan prev) memungkinkan setiap simpul untuk saling mengenali posisi tetangganya, sehingga memudahkan proses manipulasi data seperti penyisipan dan penghapusan di berbagai posisi tanpa harus mengulang penelusuran dari awal. Secara keseluruhan, pemahaman yang tepat mengenai mekanisme linking dan unlinking antar simpul menjadi faktor penentu dalam membangun sistem penyimpanan data yang stabil dan bebas dari kesalahan referensi memori.

TUGAS PRAKTIKUM

Kode Program

Kelas Lagu_2511532008

```
package pekan6_2511532008;

public class Lagu_2511532008 {
    private String judul_2008;
    private String penyanyi_2008;
    Lagu_2511532008 next_2008;
    Lagu_2511532008 prev_2008;

    public Lagu_2511532008(String judul_2008, String penyanyi_2008) {
        this.judul_2008 = judul_2008;
        this.penyanyi_2008 = penyanyi_2008;
        this.next_2008 = null;
        this.prev_2008 = null;
    }

    public String getJudul_2008() {
        return judul_2008;
    }

    public void setJudul_2008(String judul_2008) {
        this.judul_2008 = judul_2008;
    }

    public String getPenyanyi_2008() {
        return penyanyi_2008;
    }

    public void setPenyanyi_2008(String penyanyi_2008) {
        this.penyanyi_2008 = penyanyi_2008;
    }
}
```

Kelas Driver Musik_2511532008

```
public class Musik_2511532008 {
    private Lagu_2511532008 head_2008;
    private Lagu_2511532008 tail_2008;
    public void tambahLagu_2008(String judul_2008, String penyanyi_2008) {
        Lagu_2511532008 laguBaru_2008 = new Lagu_2511532008(judul_2008, penyanyi_2008);
        if (head_2008 == null) {
            head_2008 = tail_2008 = laguBaru_2008;
        } else {
            tail_2008.next_2008 = laguBaru_2008;
            laguBaru_2008.prev_2008 = tail_2008;
            tail_2008 = laguBaru_2008;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }
    public void hapusLaguAwal_2008() {
        if (head_2008 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        if (head_2008 == tail_2008) {
            head_2008 = tail_2008 = null;
        } else {
            head_2008 = head_2008.next_2008;
            head_2008.prev_2008 = null;
        }
        System.out.println("Lagu pertama berhasil dihapus!");
    }
    public void tampilMaju_2008() {
        if (head_2008 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511532008 temp_2008 = head_2008;
        System.out.println("=== Daftar Lagu (Maju) ===");
        while (temp_2008 != null) {
            System.out.println("- " + temp_2008.getJudul_2008() + " (" + temp_2008.getPenyanyi_2008() + ")");
            temp_2008 = temp_2008.next_2008;
        }
    }
}
```

```
    public void tampilMundur_2008() {
        if (tail_2008 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511532008 temp_2008 = tail_2008;
        System.out.println("=== Daftar Lagu (Mundur) ===");
        while (temp_2008 != null) {
            System.out.println("- " + temp_2008.getJudul_2008() + " (" + temp_2008.getPenyanyi_2008() + ")");
            temp_2008 = temp_2008.prev_2008;
        }
    }

    public void cariLagu_2008(String cari_2008) {
        if (head_2008 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511532008 temp_2008 = head_2008;
        boolean ketemu_2008 = false;
        while (temp_2008 != null) {
            if (temp_2008.getJudul_2008().equalsIgnoreCase(cari_2008)) {
                System.out.println("Lagu ditemukan: " + temp_2008.getJudul_2008() +
                    " oleh " + temp_2008.getPenyanyi_2008());
                ketemu_2008 = true;
                break;
            }
            temp_2008 = temp_2008.next_2008;
        }
        if (!ketemu_2008) {
            System.out.println("Lagu tidak ditemukan.");
        }
    }
}
```

```

public static void main(String[] args) {
    Scanner input_2008 = new Scanner(System.in);
    Musik_2511532008 playlist_2008 = new Musik_2511532008();
    int pilihan_2008;

    do {
        System.out.println("=== Playlist Musik NIM: 2511532008 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_2008 = input_2008.nextInt();
        input_2008.nextLine();

        switch (pilihan_2008) {
            case 1:
                System.out.print("Judul: ");
                String judul_2008 = input_2008.nextLine();
                System.out.print("Penyanyi: ");
                String penyanyi_2008 = input_2008.nextLine();
                playlist_2008.tambahLagu_2008(judul_2008, penyanyi_2008);
                break;
            case 2:
                playlist_2008.hapusLaguAwal_2008();
                break;
            case 3:
                playlist_2008.tampilMaju_2008();
                break;
            case 4:
                playlist_2008.tampilMundur_2008();
                break;
            case 5:
                System.out.print("Masukkan Judul yang dicari: ");
                String cari_2008 = input_2008.nextLine();
                playlist_2008.cariLagu_2008(cari_2008);
                break;
            case 6:
                System.out.println("Terima kasih!");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan_2008 != 6);
}

```

Contoh Output

```
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Bye
Penyanyi: Ariana Grande
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Fall From The Sky
Penyanyi: Romance Planet
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Love Me Not
Penyanyi: Ravyn Lenae
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
```



```
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Daftar Lagu (Maju) ===
- Bye (Ariana Grande)
- Fall From The Sky (Romance Planet)
- Love Me Not (Ravyn Lenae)
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
=== Daftar Lagu (Mundur) ===
- Love Me Not (Ravyn Lenae)
- Fall From The Sky (Romance Planet)
- Bye (Ariana Grande)
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan:
```

```
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus!
=== Playlist Musik NIM: 2511532008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Daftar Lagu (Maju) ===
- Fall From The Sky (Romance Planet)
- Love Me Not (Ravyn Lenae)
```